

# RaptorRuntime și RaptorJS

Roadmap complet pentru runtime, framework și aplicații web, desktop și mobile

Acest document transformă direcția Raptor într-un plan de produs și implementare. Obiectivul este un ecosistem original, instalabil independent, care include RaptorRuntime ca bază nativă, RaptorJS pentru interfețe și RaptorForge pentru inițializarea proiectelor pe mai multe platforme.

| Ce construim     | Rol                                                                      |
|------------------|--------------------------------------------------------------------------|
| RaptorRuntime    | Runtime nativ pentru JavaScript și TypeScript, separat de Node.js.       |
| RaptorJS         | Framework pentru interfețe reactive, servere și aplicații.               |
| RaptorForge      | Utilitar de creare și configurare a proiectelor web, desktop și mobile.  |
| Adaptoare native | Host-uri WebView, packaging, lifecycle și acces controlat la dispozitiv. |

## 1 Principii de produs

- Nicio copie de cod, denumire sau internals Node.js; folosim standarde publice și implementare proprie.
- Compatibilitatea este un pod selectiv spre npm, nu o promisiune de emulare a tuturor comportamentelor legacy.
- Securitatea este implicită: fișiere, rețea, mediu și procese cer capabilități declarate.
- Pachetele actuale @raptor rămân stabile; noile componente sunt adăugate fără suprascriere de comenzi.

## 2 Starea actuală

| Componentă                                  | Stare        | Ce există acum                                                         |
|---------------------------------------------|--------------|------------------------------------------------------------------------|
| @raptor/core și @raptor/dom                 | Disponibil   | Reactivitate și runtime DOM.                                           |
| @raptor/run, @raptor/server, @raptor/bundle | Disponibil   | Aplicații, server, SSR și bundling.                                    |
| RaptorRuntime                               | Specificație | Arhitectură, securitate, distribuție și roadmap definite.              |
| @raptor/forge                               | Disponibil   | Ținte web, desktop, mobile și comanda raptor-create.                   |
| Host-uri desktop și mobile                  | Planificat   | Doar granițele de integrare sunt generate; adaptoarele native urmează. |

### 3 Etapele de implementare

| Etapă                   | Rezultat                                                    | Criteriu de ieșire                                                   |
|-------------------------|-------------------------------------------------------------|----------------------------------------------------------------------|
| 0. Fundație             | Contracte, convenții de nume și testele de izolare          | Nicio comandă existentă nu este afectată.                            |
| 1. Runtime minim        | Binar nativ care rulează un modul TypeScript local          | Pornește fără Node.js instalat și oprește curat procesul.            |
| 2. Module și permisiuni | ESM, raptor.runtime.json, broker de capabilități            | Accesul neautorizat la fișiere și rețea este refuzat.                |
| 3. Servicii platformă   | HTTP, stream-uri, workers, loguri, tracing                  | Exemplu de server Raptor rulează pe runtime nativ.                   |
| 4. Pachete și build     | Manifest, lockfile, registry adapter și build reproductibil | O aplicație este împachetată identic pe sisteme suportate.           |
| 5. Desktop              | Host WebView pentru Windows, macOS și Linux                 | Aplicație semnată, lifecycle și stocare locală funcționale.          |
| 6. Mobile               | Host WebView pentru Android și iOS                          | Aplicație mobilă are navigare, lifecycle și capabilități controlate. |
| 7. Public preview       | Instalatoare, documentație și canal de actualizări          | Audit de securitate, licențe și performanță finalizat.               |

### 4 Arhitectura ecosistemului

RaptorRuntime este stratul de sistem. RaptorJS rămâne stratul de aplicație, iar RaptorForge este punctul de intrare pentru echipe. Host-urile native nu expun direct sistemul de operare către JavaScript; toate cererile trec prin brokerul de capabilități.

| Strat      | Responsabilitate                                                | Pachete sau produse                                          |
|------------|-----------------------------------------------------------------|--------------------------------------------------------------|
| Aplicație  | Ecrane, stare, rutare, server și UI                             | @raptor/core, @raptor/dom, @raptor/run, @raptor/server       |
| Unelte     | Inițializare, build, teste, profile și packaging                | @raptor/forge, @raptor/bundle, @raptor/test, @raptor/profile |
| Runtime    | Module, task-uri, permisiuni, fișiere, rețea și observabilitate | RaptorRuntime și modulele raptor:*                           |
| Host nativ | Fereastră, WebView, lifecycle, semnare și distribuție           | Raptor Desktop Adapter și Raptor Mobile Adapter              |

### 5 Fluxul pentru dezvoltatori

În prezent, RaptorForge generează proiecte fără să instaleze pachete sau să modifice configurația globală. Pentru a evita conflictul cu binarele existente, comanda este raptor-create. După stabilizarea runtime-ului, aceasta poate deveni un subcomandă raptor create printr-o migrare controlată.

| A acțiune                | Comandă                                    | Rezultat                                     |
|--------------------------|--------------------------------------------|----------------------------------------------|
| Listează platforme       | raptor-create targets                      | web, desktop, mobile                         |
| Creează aplicație web    | raptor-create site --target web            | Sursă TSX, HTML, manifest și politici.       |
| Creează desktop          | raptor-create desktop-app --target desktop | Aceleași fișiere plus granița hosts/desktop. |
| Creează mobil            | raptor-create phone-app --target mobile    | Aceleași fișiere plus granița hosts/mobile.  |
| Dezvoltă și construiește | raptor-bundle dev/build                    | Bundle pentru host-ul selectat.              |

## 6 Desktop și mobile

### Desktop

Primul adaptor desktop trebuie să aibă un nucleu comun în Rust și back-enduri de WebView pentru Windows, macOS și Linux. Responsabilitățile includ creare de ferestre, meniuri, lifecycle, deep links, notificări, stocare locală, actualizări și semnare. JavaScript primește doar API-uri raptor: și capabilitățile acordate.

### Mobile

Primul adaptor mobile trebuie să aibă un bridge minimal pentru Android și iOS, cu WebView și module native strict delimitate. Etapa inițială acoperă navigare, stocare securizată, lifecycle, deep links și comunicare cu host-ul. Camera, locația, notificările și fișierele sunt module opționale, fiecare cu capabilitate proprie.

| Capabilitate         | Desktop  | Mobile   | Politică                                   |
|----------------------|----------|----------|--------------------------------------------|
| Stocare aplicație    | Da       | Da       | Limitată la directorul aplicației.         |
| Rețea                | Da       | Da       | Allowlist de domenii și timeouts.          |
| Ferestre și navigare | Da       | Nu       | Controlată de adaptor.                     |
| Camera și locație    | Opțional | Opțional | Permișiune explicită de sistem și runtime. |
| Subprocese           | Opțional | Nu       | Doar desktop, cu listă explicită.          |

## 7 Ce mai trebuie adăugat

| Element                      | Prioritate   | Motiv                                                       |
|------------------------------|--------------|-------------------------------------------------------------|
| runtime-native               | Critică      | Hostul Rust, motorul JavaScript și launcher-ul standalone.  |
| @raptor/runtime-bridge       | Critică      | Contract stabil între RaptorJS și RaptorRuntime.            |
| raptor.runtime.json schema   | Critică      | Manifest, endpoint, țintă și capabilități verificabile.     |
| Lockfile și registry adapter | Ridicată     | Build-uri reproduse și pachete cu integritate verificabilă. |
| Raptor Desktop Adapter       | Ridicată     | Prima cale spre aplicații native pentru PC.                 |
| Raptor Mobile Adapter        | După desktop | Extinde modelul de capabilități spre telefoane.             |
| CI pentru instalatoare       | Ridicată     | Semnare, smoke tests și distribuție sigură.                 |

## 8 Ordinea recomandată

Începem cu runtime-ul minim și schema de manifest. Apoi conectăm un exemplu existent RaptorJS prin runtime bridge. Desktop urmează înaintea mobilului, deoarece oferă feedback mai rapid și acoperă instalație, packaging și capabilități locale. Mobile vine după ce contractele host-ului sunt stabile.

- Construiește spike-ul RaptorRuntime și contractele sale de test.
- Definește raptor.runtime.json și @raptor/runtime-bridge.
- Rulează un exemplu RaptorJS pe runtime-ul nativ.
- Construiește adaptorul desktop cu o aplicație demo.
- Extinde RaptorForge cu packaging și apoi cu adaptorul mobil.